

SOFTWARE REQUIREMENTS SPECIFICATION

RETAIL MANAGEMENT & INVENTORY SYSTEM

Java Full-Stack Web Application

Document Item	Details
Project Title	Retail Management & Inventory System
Document	Software Requirements Specification
Frontend	Angular
Backend	Java / Spring Boot
Database	MySQL
Communication	REST API / HTTP
Security	JWT Authentication and Role-Based Authorization
Version	1.0
Date	30 September 2026

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	3
1.1. Problem Introduction	3
1.1.1. Motivation	3
1.1.2. Project Objective	3
1.1.3. Scope of the Project	3
1.2. Organization of the SRS.....	3
CHAPTER 2 SOFTWARE REQUIREMENT SPECIFICATION	4
2.1. Product Perspective	4
2.1.1. System Interfaces	4
2.1.2. User Interface	4
2.1.3. Hardware Interfaces	4
2.1.4. Software Interfaces.....	4
2.1.5. Communications Interfaces.....	5
2.1.6. Memory Constraints	5
2.1.7. Operations	5
2.2. Product Functions.....	5
2.3. User Characteristics	5
2.4. Constraints.....	6
2.5. Assumptions and Dependencies.....	6
2.6. Non-Functional Requirements	6
2.7. Use Case.....	7
2.7.1. Use Case Model	7
2.7.2. Use Case Diagram.....	7
2.7.3. Use Case Scenario.....	8
2.8. Functional Requirements Traceability	8
2.9. Error and Status Handling	9
CHAPTER A GLOSSARY.....	10

CHAPTER 1

INTRODUCTION

The Retail Management & Inventory System is a web-based application designed to support the routine operations of a retail store. The purpose of this chapter is to introduce the problem addressed by the project, explain the motivation and objectives, define the scope, and describe the organization of this SRS.

1.1. Problem Introduction

Retail stores manage a large amount of product, stock, supplier, customer, purchase and sales information. When these records are maintained manually or across disconnected spreadsheets, it becomes difficult to keep inventory accurate and to retrieve information quickly. Manual calculations can also cause stock mismatches, duplicate records and delays in identifying low-stock products.

The proposed Retail Management & Inventory System provides a centralized platform for managing these activities. It combines an Angular-based user interface with a Java/Spring Boot REST backend and MySQL database so that business data can be stored and processed consistently.

1.1.1. Motivation

The motivation for the project is to reduce repetitive manual work and provide a practical digital solution for retail operations. A centralized inventory system can help authorized users view current stock, record purchases and sales, maintain product information and manage supplier and customer records from one application.

1.1.2. Project Objective

- To develop a secure web-based application for retail and inventory management.
- To provide authentication and role-based access for different types of users.
- To maintain accurate product and inventory information.
- To record purchase and sales transactions and update stock accordingly.
- To provide management of categories, suppliers and customers.
- To provide dashboard information and searchable records for operational use.
- To build the application using maintainable Angular, Spring Boot and MySQL components.

1.1.3. Scope of the Project

The project covers the core functions required for a retail management and inventory system. The functional scope includes user authentication, role-based authorization, product and category management, inventory monitoring, supplier management, customer management, purchase management, sales management, search/filter functionality and dashboard information.

The technical scope consists of an Angular frontend, Java/Spring Boot backend, REST APIs and a MySQL relational database. The application is intended primarily for browser-based use on desktop and laptop systems. Advanced features such as barcode scanning, multi-branch inventory and advanced analytics may be considered as future enhancements.

1.2. Organization of the SRS

- Chapter 1 introduces the project, its problem, motivation, objectives and scope.
- Chapter 2 presents the Software Requirements Specification, including product perspective, interfaces, functions, constraints, non-functional requirements and use cases.
- Appendix A provides a glossary of important terms used in the document.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION

2.1. Product Perspective

The Retail Management & Inventory System is an independent web-based software product. It follows a client-server architecture in which the Angular client communicates with a Java/Spring Boot backend through REST APIs. The backend contains the business logic, security and data-access components, while MySQL provides persistent storage.

2.1.1. System Interfaces

- Web Application Interface: The Angular application provides the user-facing pages, forms, tables, navigation and dashboard.
- Backend API Interface: Spring Boot exposes REST endpoints through which the frontend performs authentication and business operations.
- Database Interface: The backend communicates with MySQL to store and retrieve application records.
- Authentication Interface: Protected resources use the implemented JWT-based authentication mechanism.

2.1.2. User Interface

The user interface shall provide a login page, dashboard, navigation controls, forms for data entry, tables for record display, search and filter controls, validation messages and success/error feedback. Functions that are not intended for the current role may be hidden in the interface, but access control shall also be enforced by the backend.

2.1.3. Hardware Interfaces

- Client Device: Standard desktop or laptop capable of running a modern web browser.
- Processor: Intel Core i3 or equivalent or better is recommended for development and normal use.
- RAM: Minimum 4 GB recommended.
- Display: Minimum 1366 × 768 resolution recommended for comfortable dashboard and table use.
- Server: A standard development or cloud server capable of running Java/Spring Boot and MySQL.

2.1.4. Software Interfaces

Table 2.1.4: Software Interfaces

Feature	Selected Technology	Alternative	Reason for Selection
Frontend Framework	Angular	React / Vue	Component-based architecture and structured frontend development.
Backend Framework	Spring Boot	Node.js / Express	Suitable Java ecosystem for REST APIs and layered application design.
Database	MySQL	MongoDB	Relational structure is suitable for products, inventory and transactions.
Authentication	JWT	Session-based auth	Suitable for stateless REST API authentication.
API Testing	Postman	Insomnia	Convenient testing and verification of REST endpoints.

2.1.5. Communications Interfaces

- HTTP/HTTPS shall be used for communication between the Angular frontend and backend REST services.
- JSON shall be used as the primary data exchange format.
- HTTPS is recommended for deployed environments to protect credentials and authentication tokens during transmission.

2.1.6. Memory Constraints

The application should operate on standard development and client hardware. Memory requirements depend on browser usage, the size of returned records and the backend/database workload. Large record sets should be filtered or paginated where necessary to avoid unnecessary memory consumption.

2.1.7. Operations

- User Access: Users shall log in before accessing protected functionality.
- Database Operation: Application data shall be stored in MySQL.
- Backup: Database backup should be performed according to the deployment environment's operational policy.
- Session/Token Management: Authentication tokens shall be handled according to the implemented JWT security configuration.

2.2. Product Functions

The system functionality is divided into major operational modules.

Table 2.2: Product Functions

Module	Major Functions
Authentication	Registration/login as implemented, credential validation, JWT handling and logout.
User Management	Manage user accounts and roles according to administrator permissions.
Product Management	Create, view, update and delete product records.
Category Management	Create, view, update and delete product categories.
Inventory Management	View stock, update quantities through transactions and identify low-stock products.
Supplier Management	Maintain supplier details and purchase-related information.
Customer Management	Maintain customer details used for retail sales.
Purchase Management	Record purchases and increase corresponding stock.
Sales Management	Record sales, validate available stock and decrease corresponding stock.
Dashboard	Display permitted summaries of products, inventory and transaction information.

2.3. User Characteristics

- Administrator: Technically capable user responsible for system administration, user management and authorized business operations.
- Manager: User responsible for monitoring and managing retail operations, inventory and transaction-related activities.
- Employee: User who performs routine operational activities according to assigned permissions.
- Users should have basic computer knowledge and understand the retail data they are entering.

2.4. Constraints

- Browser compatibility depends on the supported Angular/browser environment.
- The application depends on the availability of the Spring Boot backend and MySQL database.
- Only authenticated users can access protected resources.
- Role restrictions must be enforced by the backend and cannot rely only on frontend controls.
- Database credentials and security secrets must be protected from exposure in source code.
- Internet/network access may be required when the application is deployed as separate frontend and backend services.

2.5. Assumptions and Dependencies

- It is assumed that users provide valid and accurate business information.
- It is assumed that the MySQL database is configured and reachable by the backend.
- It is assumed that the frontend has the correct backend API configuration.
- The system depends on Java/JDK, Spring Boot, Angular, Node.js/npm and MySQL components used by the implementation.
- It is assumed that users have sufficient permissions for the operations they attempt.

2.6. Non-Functional Requirements

Table 2.6: Non-Functional Requirements

Category	Requirement
Performance	Common user operations should respond within an acceptable time under expected project workload.
Reliability	Successful purchase and sales operations shall maintain consistent inventory information.
Security	Passwords shall not be stored in plain text; protected APIs shall enforce authentication and authorization.
Scalability	The architecture should allow additional users, products and modules without major redesign.
Usability	The interface shall provide clear navigation, labels and understandable validation/error messages.
Maintainability	Frontend, backend services and database operations shall be organized into modular components.
Availability	The system shall be accessible whenever required application and database services are running.
Portability	The frontend shall operate on supported modern browsers and the backend shall run on supported Java environments.
Data Integrity	Required fields, relationships and business rules shall be validated before data is persisted.

2.7. Use Case

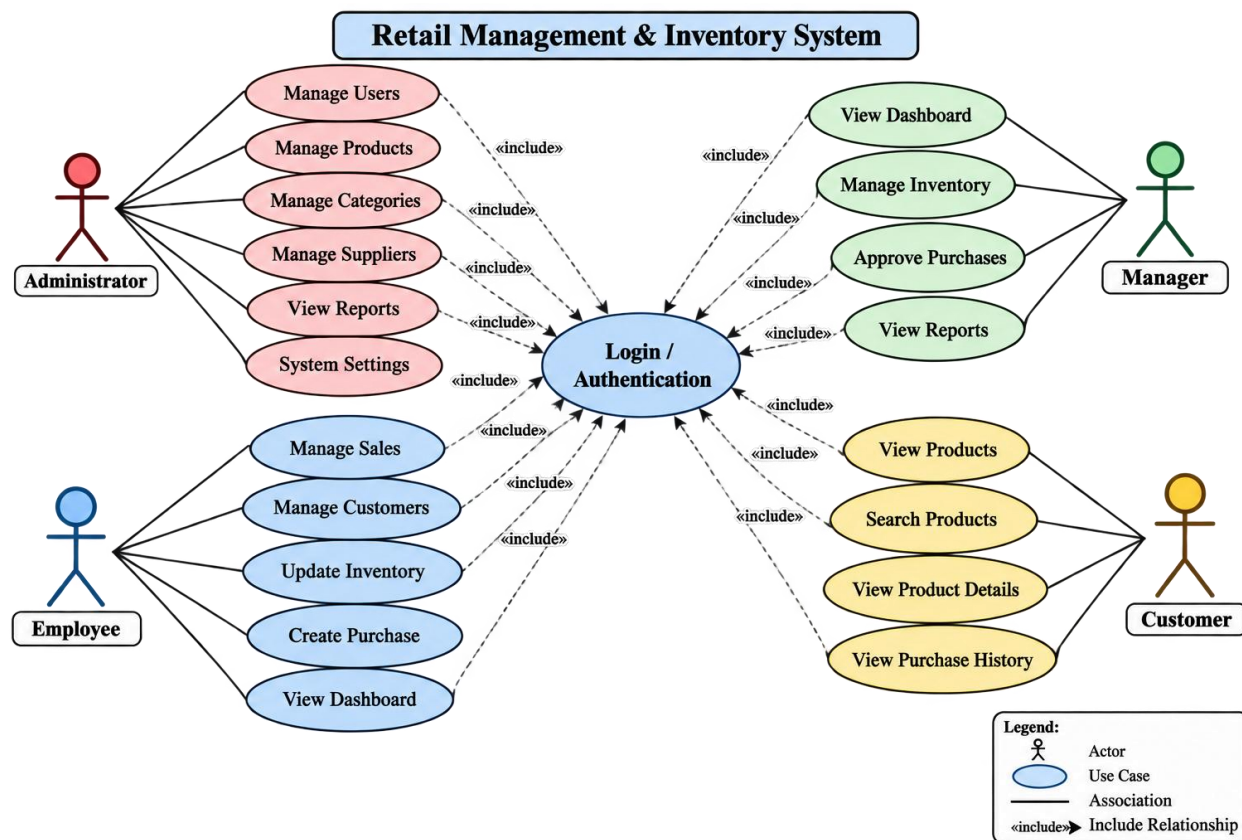
2.7.1. Use Case Model

The primary actors are Administrator, Manager and Employee. The actors interact with the Retail Management & Inventory System through authentication, product and category management, inventory management, supplier and customer management, purchase processing, sales processing and dashboard functions.

Table 2.7.1: Use Case Summary

Actor	Major Use Cases
Administrator	Login, Manage Users, Manage Products, Manage Categories, Manage Inventory, Manage Suppliers, Manage Customers, Record Purchases, Record Sales, View Dashboard, Logout.
Manager	Login, Manage Products, Manage Inventory, Manage Suppliers, Manage Customers, Record Purchases, Record Sales, View Dashboard, Logout.
Employee	Login, View/Manage permitted Products, View Inventory, Manage permitted Customers, Record permitted Sales, View permitted Dashboard information, Logout.

2.7.2. Use Case Diagram



[Use Case Diagram – Retail Management & Inventory System]

Administrator / Manager / Employee → Authentication, Products, Inventory, Suppliers, Customers, Purchases, Sales, Dashboard

2.7.3. Use Case Scenario

Table 2.7.3: Use Case Scenario

Use Case Element	Description
Use Case Number	UC-08
Use Case Name	Record Sale
Primary Actor	Authorized Employee / Manager / Administrator
Precondition	User is authenticated and has permission to record a sale; selected product exists.
Trigger	User submits a sales transaction.
Basic Flow	1. User selects customer when applicable. 2. User selects product(s) and quantities. 3. System validates required data. 4. System checks available stock. 5. System stores the sale. 6. System decreases inventory. 7. System returns success.
Alternate Flows	A1: Invalid input — system displays validation error and does not save the transaction. A2: Insufficient stock — system rejects the sale and inventory remains unchanged. A3: Server/database failure — system returns an error response.
Postcondition	A successful sale is stored and the corresponding inventory quantity is updated.

2.8. Functional Requirements Traceability

Table 2.8: Functional Requirements Traceability

Requirement ID	Module	Requirement Statement
FR-01	Authentication	The system shall authenticate users using valid credentials.
FR-02	Authorization	The system shall restrict protected operations according to role.
FR-03	Products	Authorized users shall perform CRUD operations on products.
FR-04	Categories	Authorized users shall manage product categories.
FR-05	Inventory	The system shall maintain current stock quantities.
FR-06	Suppliers	Authorized users shall manage supplier records.
FR-07	Customers	Authorized users shall manage customer records.
FR-08	Purchases	The system shall record purchases and increase stock.
FR-09	Sales	The system shall record sales and decrease stock after validation.
FR-10	Validation	The system shall reject invalid or incomplete input.

FR-11	Search	The system shall support search/filter operations where implemented.
FR-12	Dashboard	The system shall display permitted operational summaries.
FR-13	Logout	Authenticated users shall be able to terminate access.

2.9. Error and Status Handling

Table 2.9: API Status and Error Handling

HTTP Status	Meaning	Example
200 OK	Successful operation	Record retrieved/updated successfully.
201 Created	New resource created	Product/customer created.
400 Bad Request	Invalid request	Required field missing or invalid quantity.
401 Unauthorized	Authentication failure	Missing or invalid JWT.
403 Forbidden	Permission denied	User role cannot perform requested operation.
404 Not Found	Resource unavailable	Requested product or customer does not exist.
500 Internal Server Error	Unexpected backend failure	Unhandled server/database error.

CHAPTER A

GLOSSARY

Table A.1: Glossary

Term	Definition
Inventory	Current quantity and availability of products maintained by the store.
Stock	Quantity of a product available for sale.
JWT	JSON Web Token used for token-based authentication.
REST API	HTTP-based interface used for communication between application components.
RBAC	Role-Based Access Control, in which permissions depend on a user's assigned role.
CRUD	Create, Read, Update and Delete operations.
Transaction	A recorded business operation such as a purchase or sale.
SRS	Software Requirements Specification.